SolarSight

The Complete Beginner's Guide

Every question an interviewer could ask — answered from zero

Thermal solar-panel fault detection with YOLO11, Streamlit & Supabase

Project by Lalit Ranjan Behera · Guide version 1.0 · September 2026

All metrics in this guide come from runs actually performed on this machine (CPU-only).

Nothing here is invented — items that were **not** tested are labelled honestly.

Contents

1. How to use this guide
2. The project in one story
3. Every concept you need (zero-knowledge vocabulary)
4. Architecture — how the pieces fit
5. The dataset — what we trained on
6. Training — what actually happened
7. Evaluation — reading the numbers honestly
8. Inside the app — module by module
9. The database — Supabase & PostgreSQL
10. Testing & CI — how we know it works
11. Docker & deployment
12. Roadmap — what comes next
13. Interview Q&A bank (45 questions)
14. Known limitations — the honest list
15. Cheat sheet — every number you must remember
16. Glossary quick reference

1. How to use this guide

This guide assumes **zero prior knowledge**. It is written for someone who has never trained a model, never used a database, and never deployed an app. Read it top to bottom the first time; later, jump to a section before an interview. Three kinds of callouts appear throughout:

- **WHY** — the reasoning behind a design choice (interviewers love asking "why?").
- **HONEST** — a limitation or something that was *not* tested. Never oversell these in an interview.
- **TIP** — a memory hook or a likely interview follow-up.

> **Golden rule of this project:** every number in this guide was actually measured on this machine. If a thing was not run (Docker build, GPU training, live Supabase round-trip), the guide says so explicitly. In interviews, this honesty is a strength, not a weakness.

2. The project in one story

> **The story (memorise this 60-second version):** Solar panels develop hot defects — broken bypass diodes, faulty strings — that are invisible to the naked eye but show up as hot spots in **thermal (infrared) images**. Today, technicians inspect panels by carrying a thermal camera and judging images by eye: slow, inconsistent, and easy to get wrong. SolarSight is a web application where an inspector uploads a thermal image (or video), a trained AI model called **YOLO11n** finds and labels the faults in under a second on a normal laptop CPU, draws boxes around them with a confidence score, and saves a record of the inspection to a cloud database. A **History screen** shows past inspections with charts and a CSV export for reports. The project was built end-to-end on a CPU-only laptop: dataset from Kaggle, training, evaluation, testing (33 automated tests), documentation, and deployment packaging.

What the user actually sees

- **Upload screen:** a sidebar (site name, confidence slider, IoU slider, screen picker) and an image/video uploader. After analysis: the annotated image with coloured boxes, a table of detections (class + confidence), and a "Save inspection" button.
- **History screen:** past inspections with filters (site, media type), charts of fault types, drill-down into any inspection, and one-click CSV export.

The 8 fault classes the model detects

These names come straight from the dataset. In plain English:

Class	What it means physically
`MultiByPassed`	Several bypass diodes are shorted/active — usually a whole sub-string of cells running hot.
`MultiDiode`	Several individual diode defects — multiple small hot patches on one panel.
`MultiHotSpot`	Multiple hot spots — cells overheating, e.g. from shading or soiling.
`SingleByPassed`	One bypass diode bypassed — a distinct hot band pattern.
`SingleDiode`	One diode fault — a single localised hot patch.
`SingleHotSpot`	One hot spot — a single overheating cell or area.
`StringOpenCircuit`	A string (chain) of panels has an open circuit — characteristic whole-string pattern.
`StringReversedPolarity`	A string wired with reversed polarity — a dangerous wiring fault.

> **TIP:** the names encode *count* (Single/Multi/String) + *fault type* (Bypassed/Diode/HotSpot/OpenCircuit/ReversedPolarity). If an interviewer asks "what is a bypass diode?": it is a safety diode wired in parallel with a group of cells; when cells are shaded or damaged it gives current an alternate path so the panel does not burn out — but if the diode itself fails, part of the panel runs hot, which is exactly what thermal cameras see.

3. Every concept you need (zero-knowledge vocabulary)

Each entry: what it is in one or two sentences, then why this project uses it.

3.1 AI and machine learning basics

Term	Explanation
Artificial Intelligence (AI)	Software that performs tasks that normally need human intelligence. Machine learning is one way of building AI.
Machine Learning (ML)	Instead of writing rules by hand, we show a program many examples and it *learns* the rules. SolarSight shows a network thousands of thermal images with boxes drawn around faults; it learns what faults look like.
Supervised learning	Learning from examples that already have the correct answer attached (labels). Our dataset has boxes with class names — that is supervision. Opposite: unsupervised (no labels).
Classification vs. detection	Classification answers "what is in this image?" (one label). **Object detection** answers "what is here, and exactly where?" — a list of (class, box, confidence). SolarSight needs detection, because an image can contain several different faults at known positions.
Neural network	A stack of simple mathematical units ("neurons") organised in layers. Each layer transforms its input a little; stacked together they can recognise complex patterns. "Deep learning" = many layers.
CNN (Convolutional Neural Network)	A network whose layers slide small filters across the image to detect local patterns (edges → textures → parts → objects). This matches how images are structured, so CNNs dominate vision tasks. YOLO is a CNN-based detector.
Weights (parameters)	The learned numbers inside the network. "Training" = adjusting weights so predictions match labels. Our trained model is one file, `models/best.pt`, about 5.4 MB of weights.
Inference	Using a trained model to predict on new data. Training happens once; inference happens every time the app analyses an upload. Measured on this CPU: **76 ms/image** for the nano model at 640 px.
GPU vs CPU	A GPU has thousands of small cores — ideal for the maths of training. This machine has **no GPU**, so training ran on the 8-core CPU, which is why we trained a small model at a modest resolution for few epochs (Part 6). Inference is fast enough on CPU.

3.2 YOLO and object detection

Term	Explanation
YOLO	"You Only Look Once" — a detector that predicts all objects in **one pass** of the network (older methods ran the network thousands of times per image). One pass = fast, which is why YOLO is the industry standard for real-time detection. We use **YOLO11n** ("nano" = the smallest, fastest variant of the YOLO11 family) via the `ultralytics` library.
Bounding box	The rectangle drawn around a detected object. Stored as x1, y1 (top-left) and x2, y2 (bottom-right) pixel coordinates — exactly the columns in our `detections` table.
Confidence score	A number 0–1 for how sure the model is that a box really contains that class. We discard predictions below **0.40** (the `DEFAULT_CONF_THRESHOLD` in `src/config.py`). Lower → more detections but more false alarms; higher → safer but misses subtle faults.
NMS — Non-Maximum Suppression	The model often proposes several overlapping boxes for the same fault. NMS keeps the highest-confidence box and drops overlaps (measured by IoU above a threshold, ours **0.45**). Without NMS you would see 3 boxes on one hot spot.
Feature pyramid / multi-scale heads	YOLO detects at three output scales (small/medium/large boxes) so both tiny hot spots and whole-string faults are caught. (Concept question only — nothing to configure.)
Transfer learning	Starting from a model already trained on a huge dataset (COCO: 80 everyday classes) instead of random weights. We fine-tuned pretrained `yolo11n.pt` on thermal images. WHY: thermal faults need far fewer examples to learn when the network already "knows" edges, textures and objectness — training is faster and needs less data.
Object tracking (ByteTrack)	Detection works per-frame. Tracking links detections across video frames so the same physical fault keeps the same **track ID**. Ultralytics has ByteTrack built in; we call `model.track(persist=True)` — persist keeps tracker memory between frames. Used in `src/pipelines/video.py`.

3.3 Training vocabulary

Term	Explanation
Dataset split	Train (the model learns from it), Validation (checkpoint metrics during training), Test (final honest exam, touched once). Our split: **6,924 / 400 / 15** images. WHY a test set: measuring on training data is like grading a student on the homework they memorised — the score would be meaningless.
Epoch	One full pass over the training set. We trained **10 epochs**.
Batch size	Images processed together before weights update. We used **batch 8**. Bigger batches need more RAM; this laptop has 7.4 GB.
Learning rate	Step size for weight updates. Too high → training oscillates/diverges; too low → painfully slow. Ultralytics picked a sensible schedule; you can see it decay in `docs/assets/training_results.csv`.
Loss function	The number training tries to minimise: how wrong the predictions are. YOLO sums three parts: **box loss** (are the rectangles in the right place?), **classification loss** (are the labels right?), **DFL loss** (distribution focal loss — refined box-edge quality). All three decrease across our 10 epochs.
Backpropagation / optimizer	Backprop computes how much each weight contributed to the loss; the optimizer (SGD/AdamW family) nudges weights the other way. Repeat millions of times. (Concept-level answer is enough.)
Overfitting	The model memorises training images but fails on new ones. Symptom: training loss keeps dropping while *validation* metrics stall or fall. Our validation mAP rose every epoch — no overfitting within these 10 epochs.
Data augmentation	Randomised copies of training images (flips, crops, colour shifts) so the model sees variety and generalises better. Ultralytics applies it automatically. NOTE the separate dataset issue in Part 5: this dataset already *contains* augmented near-duplicates — a different thing, and a metrics risk.
Random seed	A fixed number (we used **seed 42**) that makes random choices reproducible, so re-running training gives (nearly) the same result. Interviewers ask "how would someone reproduce your run?" — seed + pinned library versions + dataset version 49.
imgsz (image size)	Network input resolution. We trained at **320 px** (not the dataset's native 640) because CPU training time roughly quadruples when you double the side length. Trade-off: small distant faults may be harder at 320.
Checkpoint / best.pt	A snapshot of weights. Ultralytics saves `best.pt` = the checkpoint with the best validation fitness. We copied it to `models/best.pt` — the single file the app loads.

3.4 Evaluation vocabulary (the metrics that decide your grade)

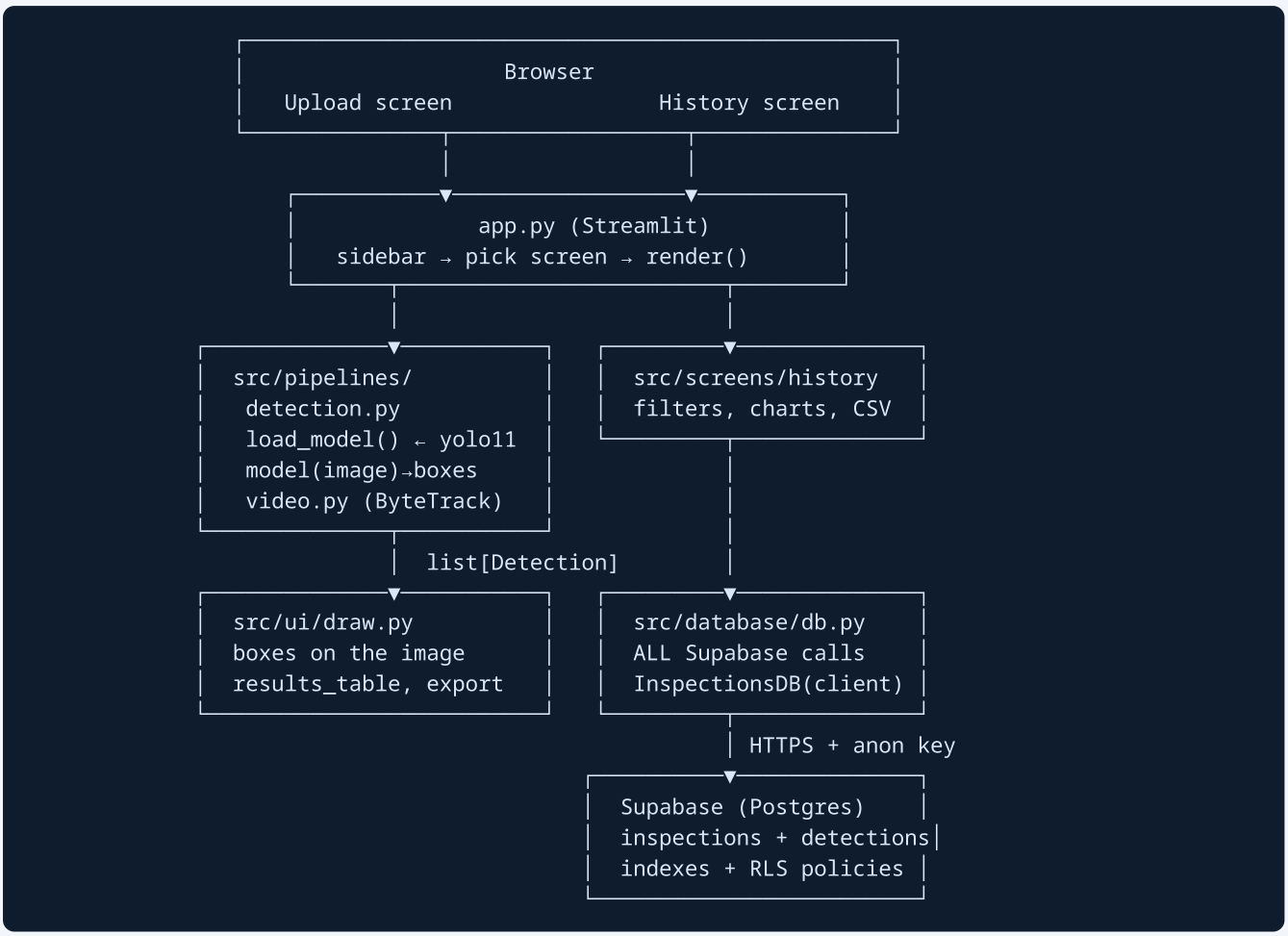
Term	Explanation
IoU — Intersection over Union	Overlap between predicted box and true box, 0–1. IoU = area of overlap ÷ area of union. A prediction "counts" as correct only if IoU ≥ the threshold (0.50 for mAP50).
Precision	Of the boxes the model drew, how many were real? precision = TP / (TP + FP). High precision = few false alarms. Our overall test precision: **0.692**.
Recall	Of the real faults, how many did the model find? recall = TP / (TP + FN). High recall = few missed faults. Our overall test recall: **0.762**.
TP / FP / FN	True positive (correct box), false positive (box on nothing / wrong class), false negative (real fault missed). Everything in detection metrics is built from these three counts.
AP — Average Precision	The area under the precision-vs-recall curve for one class: a single number summarising the precision/recall trade-off at all thresholds.
mAP50	Mean AP across classes at IoU 0.50. Our test mAP50: **0.641**. This is the headline number of the project.
mAP50-95	Mean AP averaged over IoU thresholds 0.50, 0.55, …, 0.95 — rewards *precise* boxes, not just found boxes. Harder to score high. Ours: **0.541** on test.
Confusion matrix	A grid: rows = true class, columns = predicted class. Diagonal = correct. Off-diagonal cells reveal which classes get mixed up (e.g. SingleDiode vs SingleHotSpot). Saved: `docs/assets/confusion_matrix.png`.
PR curve	Precision-vs-recall plot per class; the closer it hugs the top-right corner, the better. Saved: `docs/assets/pr_curves.png`.
F1 score	The balance point of precision and recall: F1 = 2PR/(P+R). Useful one-liner when an interviewer wants a single number per class.

3.5 Web-app, database and tools vocabulary

Term	Explanation
Streamlit	A Python framework that turns scripts into web apps — no HTML/JavaScript needed. It **re-runs your script top-to-bottom on every interaction** (a "rerun"), which is why caching matters (below). App entry point: `app.py`.
st.cache_resource	Decorator that loads something ONCE per process and shares it across all user sessions. We cache the YOLO model with it — otherwise every visitor would reload the 5.4 MB weights (seconds of waiting).
Session state	Streamlit memory per visitor that survives reruns (e.g. remembering the last analysis result).
PostgreSQL	A battle-tested open-source relational database. Data lives in **tables** (like spreadsheets with strict column types).
SQL	The language of relational databases: `INSERT` (save), `SELECT` (read), `DELETE`, etc. In this project SQL appears only in `supabase/schema.sql`; the app talks to the database over an API instead.
Supabase	A hosted Postgres service with a ready-made REST API, so Python code never writes raw SQL — it calls `.table("inspections").insert(...)` via the `supabase-py` client. Free tier is enough for a demo. This is a portfolio-app classic because it needs no server of your own.
REST API / PostgREST	Every table automatically gets HTTP endpoints. The anon key in the app identifies the request as "anonymous visitor".
Row Level Security (RLS)	Database-side rules about which rows each key may read/write. WHY it matters: anyone could extract the anon key from the app, so the *database itself* must limit what that key can do. Our schema enables RLS and adds permissive anon policies (insert/select/delete) — appropriate for a demo with no user accounts; a real product would tie policies to authenticated users.
Primary key / UUID	A unique row identifier. We use auto-generated UUIDs for `inspections.id` and `detections.id`.
Foreign key + cascade	`detections.inspection_id` points at `inspections.id`. `ON DELETE CASCADE`: deleting an inspection automatically deletes its detections — no orphan rows.
Index	A lookup structure that makes specific queries fast (like a book index). We add three: `created_at desc` (newest-first history), `inspection_id` (drill-down lookups), `class_name` (charts by fault type).
CSV	Comma-separated values — the universal export format. History screen exports the filtered table; report data is produced with pandas.
pytest	Python's standard test framework. Tests are small programs that check units of code automatically. Ours: **33 tests, all passing in ~2.5 s**.
Test double (fake/mock)	A stand-in object replacing a real dependency (like the database) so tests run offline and instantly. Our `tests/conftest.py` fakes the Supabase fluent API (chains like `.table().insert().execute()`).
Smoke test	A minimal "does it even start?" check. Two Streamlit `AppTest` smoke tests boot the app and assert the screen-picker radio exists (found by label `"Screen"`).
CI (Continuous Integration)	A robot that re-runs checks (dependencies sanity, import check, tests) on every push. GitHub Actions workflow: `.github/workflows/ci.yml`; run the same steps locally with `scripts/ci_local.sh`.

Docker	Packages app + dependencies into a **container image** that runs identically anywhere. `Dockerfile` = recipe (python:3.11-slim base, install requirements, healthcheck on Streamlit's `/_stcore/health` endpoint); `docker-compose.yml` = one-command runner.
Git	Version control — a time machine for code. `git commit` snapshots the project. This repo is on branch `main`.
Environment (conda)	An isolated Python installation so project dependencies do not clash with the system. This project uses conda env `solarsight` (Python 3.11.16) because the system Python 3.14 is not supported by torch yet.

4. Architecture — how the pieces fit



Life of an upload (follow the data)

1. Inspector enters a **site name** in the sidebar and uploads a thermal image.
2. `src/screens/upload.py` checks the file type; the model is already loaded once per process (via `@st.cache_resource` → `load_model()` in `src/pipelines/detection.py` — the only file allowed to import ultralytics).
3. The image runs through the model; raw results are converted into a list of `Detection` objects (a frozen dataclass: class_name, confidence, x1, y1, x2, y2, track_id).
4. Detections below the confidence threshold (default **0.40**) are filtered out (inclusive: 0.40 stays).
5. For video, detections are deduplicated per track (`dedupe_by_track`) so one fault = one row, not one row per frame.
6. `src/ui/draw.py` draws labelled boxes on the image; `components/results_table.py` renders the table.
7. On "Save inspection": `database/db.py::save_inspection()` inserts one `inspections` row, then the `detections` rows. If secrets are missing, the section degrades gracefully (`is_configured()` shows a setup notice instead of crashing).
8. The History screen reads rows back through the same `db.py` , filters in Postgres, aggregates chart counts, and exports CSV.

Why the code is split this way (asked constantly!)

Module	Responsibility	Why separate
`src/config.py`	Paths, thresholds, class list	Single source of truth — change 0.40 once, everywhere updates.
`src/pipelines/detection.py`	Model loading + inference + filtering	The only ultralytics import; UI never touches YOLO directly.
`src/pipelines/video.py`	Frame-by-frame tracking + summarising	Video logic is complex; keeping it out of screens keeps both testable.
`src/database/db.py`	Every Supabase call	Swap/mock the DB in one place; tests inject a fake client (dependency injection).
`src/ui/draw.py`, `components/*`	Boxes, tables, CSV export	Pure presentation — no model or DB knowledge.
`src/screens/*`	Page layouts and user flow	Compose the pieces; stay thin.
`training/*`	Download, train, evaluate	One-time/offline work, kept out of the app entirely.
`tests/`	33 automated checks	Confidence to change code without breaking it.

WHY this matters in interviews: this is a *layered architecture* (UI → pipelines → database). Each layer depends only on the one below. That is what makes the 33 tests possible: the database layer can be faked, the model layer can be faked, and the UI can be smoke-tested without a GPU, without internet, and without secrets.

5. The dataset — what we trained on

Fact	Value
Source	Kaggle dataset `pkdarabi/solarpanel` — "Thermal Solar PV Anomaly Detection Dataset" (ThermoSolar-PV), version 49.
Download method	`kagglehub==1.0.2` — anonymous, no Kaggle account needed. Script: `training/download_data.py`.
License	**CC BY-NC 4.0** — free for non-commercial use with attribution. (Interview question! It means this demo/portfolio use is fine; selling it would not be.)
Images	**7,339** JPGs, all pre-sized **640×640**, thermal (infrared) captures.
Splits	train **6,924** · valid **400** · test **15** images.
Boxes (labels)	train **19,845** · valid **1,119** · test **31** boxes.
Classes	**8** fault classes (list in Part 2), ids 0–7 exactly as in `src/config.py::CLASS_NAMES`.
Format	YOLO format: one `.txt` per image, each line `class_id x_center y_center width height` (normalised 0–1).
Missing in test	3 of 8 classes have zero ground-truth boxes in the 15 test images: `MultiByPassed`, `StringOpenCircuit`, `StringReversedPolarity`.

HONEST — the augmentation trap: the dataset contains roughly **3 augmented near-duplicate versions of each source image**, and they are spread *across* train/valid/test. That means the test set is not fully independent: the model has seen near-identical images during training. Effect: our metrics are **optimistic** (inflated). We report this caveat next to every number instead of hiding it. A proper fix would be re-splitting by source image — listed in the roadmap.

WHY 15 test images is a problem: 31 boxes across 15 images means each class is represented by only a handful of boxes. One extra miss moves a class's recall by a big jump — the per-class numbers are statistically fragile. Say this out loud in interviews; it shows you understand measurement, not just buttons.

6. Training — what actually happened

The full honest write-up lives in `docs/TRAINING.md`. The essentials:

Setting	Value & reason
Model	YOLO11n (`models/yolo11n.pt`), COCO-pretrained → transfer learning. Smallest/fastest of the family — the realistic choice for CPU-only hardware and a 5.4 MB deployable file.
Epochs / imgsz / batch / workers	**10 / 320 / 8 / 2**. Chosen for this machine: no GPU, 8-core i3-1215U, 7.4 GB RAM. Actual wall time: **~94 minutes**.
Device	`cpu` (auto-detected; `training/train_yolo.py` prints it). torch 2.14.0+cpu build.
Seed	**42** — reproducibility.
Output	`runs/detect/runs/solar/train/` → best weights copied to `models/best.pt` (5.4 MB). Learning curve saved to `docs/assets/training_results.csv` .

The real learning curve (validation mAP50 per epoch)

epoch	1	2	3	4	5	6	7	8	9	10
mAP50	.127	.195	.267	.327	.360	.442	.453	.515	.516	.554

- Steady climb, no plateau, no collapse → the model was still improving when we stopped.
- mAP50-95 reached **0.368** at epoch 10.
- Training losses (box/cls/dfl) all decreased monotonically.

WHY only 10 epochs? Time budget on CPU: ~9 min/epoch at 320 px. 10 epochs was enough to prove the pipeline works end-to-end and produce a useful model. The curve says more epochs would help — that run is documented for a free GPU via `training/colab_train_gpu.ipynb` (640 px, 60 epochs, T4). **This notebook was NOT run** — say so if asked.

> **TIP — the classic question: "Why not train at 640 like the dataset?"** Answer: compute cost. Doubling the side length roughly quadruples the pixels per image, and CPU epochs were already ~9 minutes at 320. I chose a working 320 px model now + a documented GPU path to 640 later, and I report the metrics that the shipped 320 px model actually achieves.

7. Evaluation — reading the numbers honestly

Produced by `training/evaluate.py --conf 0.40` on the untouched test split, using the exact shipped `models/best.pt`. Artifacts: `docs/assets/evaluation_metrics.{md,csv}`, `confusion_matrix.png`, `confusion_matrix_raw.png`, `pr_curves.png`, `predictions_grid.png`, plus 4 annotated sample predictions (`docs/assets/samples/`, real confidences 0.40–0.73 from `scripts/run_samples.py --count 4`).

Overall test results

precision	recall	mAP50	mAP50-95
0.692	**0.762**	**0.641**	**0.541**

Per-class results

class	precision	recall	mAP50	mAP50-95
MultiByPassed	—	—	—	— (absent from test GT)
MultiDiode	1.000	1.000	**0.995**	0.895
MultiHotSpot	0.583	1.000	0.820	0.690
SingleByPassed	0.909	0.909	0.848	0.685
SingleDiode	0.800	0.400	0.375	0.300
SingleHotSpot	0.167	0.500	**0.165**	0.132
StringOpenCircuit	—	—	—	— (absent)
StringReversedPolarity	—	—	—	— (absent)

How to tell this story in an interview (60 seconds)

1. **Headline:** "Test mAP50 is 0.641 with a 5.4 MB model running at 76 ms/image on a laptop CPU."
2. **Best classes:** "MultiDiode is near-perfect (0.995); SingleByPassed and MultiHotSpot are solid (0.85 / 0.82)."
3. **Weak classes:** "SingleHotSpot is weak (0.165) — few test instances, visually subtle, and precision 0.167 means most of its predictions are false positives. SingleDiode has the opposite profile: precise (0.80) but missing half the instances (recall 0.40)."
4. **Missing classes:** "Three classes have no ground truth in the 15-image test split, so their metrics are undefined — I print '—', never 0. A 0 would claim the model failed a test it never took."
5. **Caveat:** "The dataset's augmented near-duplicates leak across splits, so these numbers are optimistic. Fix on the roadmap: re-split by source image."

> **HONEST — a real bug we hit (great interview story):** the first version of `evaluate.py` crashed with an IndexError. Ultralytics orders per-class metric arrays by *position among classes present in the split* (`metrics.box.ap_class_index`), not by global class id — so `result[3]` is NOT necessarily class 3. Fixed by mapping `present.index(class_id)`. Lesson: never assume library internals; read the arrays' own index metadata.

8. Inside the app — module by module

`app.py` — entry point (~30 lines)

Sets the page config (title, ☀️ icon, wide layout), renders the sidebar, then imports and calls the chosen screen. Screens are imported *inside* the branch so a History visit never pays the cost of importing upload machinery, and vice versa.

`src/config.py` — one source of truth

- `MODEL_PATH` = `models/best.pt`; `DATA_DIR` = `data/`.
- `DEFAULT_CONF_THRESHOLD = 0.40` (filter is inclusive — 0.40 passes).
- `DEFAULT_IOU_THRESHOLD = 0.45` for inference-time NMS (ultralytics default is 0.7; 0.45 keeps close-together faults from being merged).
- `CLASS_NAMES`: the 8 classes, list index = trained class id.
- Accepted image / video extensions.

`src/pipelines/detection.py` — the model seam

- `Detection`: frozen dataclass = one fault (class_name, confidence, box, optional track_id). Frozen = immutable = safe to pass around.
- `load_model()`: the ONLY place importing ultralytics. UI code depends on this seam, so tests can inject fakes.
- `results_to_detections()`: converts ultralytics results into `Detection`s. Duck-typed and `boxes=None`-safe (frames with no detections happen).
- Filtering by confidence (≥ threshold), and `dedupe_by_track()` for video.

`src/pipelines/video.py` — tracking

- `iter_video_detections()`: a Python *generator* that yields (frame, detections) per frame using `model.track(persist=True)` — ByteTrack keeps track IDs stable across frames.
- `summarize_tracks()`: collapses per-frame detections into one entry per track id, with the max confidence seen.
- WHY dedupe: without it, a 10-second clip of one faulty panel would save hundreds of duplicate rows.

`src/database/db.py` — the only file that knows Supabase exists

- `InspectionsDB(client)`: the client is **injected** (passed in). Production passes the real client; tests pass a fake. This is dependency injection — expect that exact phrase in interviews.
- `is_configured()`: checks secrets exist, never raises → the UI can degrade gracefully.

- `save_inspection()`: one parent `inspections` row + bulk `detections` insert. Empty detections list = valid no-op (PostgREST rejects empty bulk inserts; zero faults is a legitimate finding).
- Reads: `list_inspections()` filters with `.eq()` **in Postgres** (not in Python — pull only the rows you show), newest first via index on `created_at`; `fault_counts()` aggregates per class in Python with a Counter (documented trade-off: PostgREST GROUP BY needs an RPC; row counts are tiny); `distinct_sites()` feeds the filter dropdown.
- `detection_row()`: maps a `Detection` to table columns — kept separate so it is unit-testable alone.

> **HONEST:** `save_inspection()` is "transactional-ish": PostgREST has no client-side transactions, so if the detections insert failed after the parent row was written, the parent would remain. Documented in TESTING.md. (Also: with no Supabase keys on this machine, the save/history round-trip has **not** been exercised live — it is covered by 33 offline tests with a fake client.)

`src/screens/upload.py` & `history.py`

- Upload: image flow (predict → draw → table → save section) and video flow (track → summarise → save). Model cached with `@st.cache_resource` — loaded once per process, shared across sessions.
- History: filters (site/media type) applied in the database, line chart + bar chart of fault counts, drill-down into one inspection, CSV export via `components/export.py`, delete-inspection support (cascade removes detections).

Streamlit mechanics worth explaining

- **Rerun model:** every widget click re-executes the script top-to-bottom. That is why the model is cached and results are kept in session state.
- **cache_resource vs cache_data:** *resource* = one shared object (the model); *data* = cache keyed by arguments (dataframes).
- **Secrets:** `.streamlit/secrets.toml` (gitignored) holds `SUPABASE_URL` / `SUPABASE_KEY`; a committed `secrets.toml.example` documents the shape.

9. The database — Supabase & PostgreSQL

`inspections` — one row per analysis run

Column	Type	Meaning
`id`	uuid PK	auto UUID, the drill-down key
`site_name`	text NOT NULL	user-entered site label (filterable)
`source_filename`	text NOT NULL	original file name
`media_type`	text	'image' or 'video' (CHECK constraint)
`model_version`	text NOT NULL	which weights produced it — provenance!
`created_at`	timestamptz	defaults to now(); indexed DESC for history
`total_faults`	integer	denormalised count for instant listing

`detections` — one row per found fault

Column	Type	Meaning
`id`	uuid PK	auto UUID
`inspection_id`	uuid FK → inspections	`ON DELETE CASCADE`
`class_name`	text	fault class (indexed for charts)
`confidence`	double precision	0–1, enforced by CHECK
`x1,y1,x2,y2`	double precision	box corners in pixels
`track_id`	integer NULL	set for video only, NULL for images

Design decisions to narrate

- **Metadata only — no images stored.** Keeps the DB small and avoids storage/privacy issues; the upload is analysed in memory and discarded.
- **Cascade delete** = no orphan detection rows ever.
- **Three indexes**, each serving one real query: newest-first history; per-inspection drill-down; per-class chart aggregation.
- **RLS on both tables** with anon policies. WHY: the anon key ships with the app, and Supabase also auto-exposes a REST API — without RLS the tables would be readable by anyone. Policies currently allow anon insert/select/delete (a demo trade-off); a production system would scope rows to authenticated users.
- **CHECK constraints** (`media_type`, confidence bounds) — the DB defends itself against bad data even if app code has a bug.

> **TIP — "Why Supabase and not SQLite/MySQL/Firebase?"** SQLite: file-based, not shareable across users/machines, no API or security layer. MySQL: fine, but Supabase gives hosted Postgres **plus** a ready REST API and RLS — zero server code. Firebase: NoSQL — poor fit for our relational inspection→detections shape and its SQL-friendly filters/charts.

10. Testing & CI — how we know it works

33 tests, all passing, ~2.5 seconds (`pytest -q`). No GPU, no internet, no secrets required.

What the suite covers

- **Config/paths:** class list matches the 8 dataset classes; thresholds sane.
- **Detection pipeline:** threshold filtering (inclusive ≥), conversion from fake ultralytics results (fakes mimic `.cpu().numpy()` — the real call chain), `boxes=None` safety, track dedupe.
- **Video:** track summarising keeps max confidence per track id.
- **Database layer:** a `FakeSupabaseClient` reproduces the fluent PostgREST chain (`.table().insert().execute()`, `.eq()`, `.in_()`, `.order().limit()`), so every `db.py` method is asserted offline — including a bug we found and fixed in the fake's `.in_()` filter during development (documented in TESTING.md).
- **UI smoke tests:** Streamlit `AppTest` boots the real app and asserts the screen-picker radio exists — **located by label "Screen"**, not by index (the Upload screen adds its own "Media type" radio, which broke index-based lookup; that failure and fix are documented).

- **Mapping:** `detection_row()` produces exactly the table's columns.

Failures we actually hit and fixed (TESTING.md lists all 8)

1. `sys.path` — training scripts couldn't import project modules when run from another directory.
2. Dataset double-nesting — kagglehub extracts an inner `ImageSet/`; naive copying nested it twice.
3. AppTest relative path — smoke test must launch `app.py` from the project root.
4. Fake Supabase `.in_()` filter returned wrong rows.
5. Missing imports in the detection module after a refactor.
6. Dead import in `upload.py` (lint found it).
7. The ultralytics per-class array ordering bug (Part 7).
8. `requirements.txt` pinned `opencv-python-headless==5.0.0`, a version that does not exist on PyPI (real release: `5.0.0.93`). It survived unnoticed because the original env already had opencv installed; a from-scratch install following BUILDING.md exposed it, and since pip is all-or-nothing the whole install aborted. Fix: correct pin + full re-verification.

> **WHY interviewers love this:** "tell me about a bug" is a standard question, and these are real, explained, fixed, regression-protected bugs — not memorised horror stories.

CI

`.github/workflows/ci.yml` runs on every push: dependency sanity (`pip check`), import-the-app-without-secrets check, and pytest. `scripts/ci_local.sh` runs the identical steps locally — all steps pass on this machine.

11. Docker & deployment

Docker in 5 sentences

1. An **image** is a frozen snapshot: OS + Python + dependencies + your code.
2. A **container** is a running instance of that image.
3. A **Dockerfile** is the recipe; ours starts from `python:3.11-slim`, installs `requirements.txt`, copies the app, and runs Streamlit on the container's port.
4. A **HEALTHCHECK** pings `/_stcore/health` (Streamlit's built-in endpoint, which returned `ok` in our live run) so orchestrators know the app is actually alive.
5. **docker-compose.yml** wraps the run command + env/secrets wiring into one `docker compose up`.

> **HONEST:** Docker is **not installed on this machine**, so the image has **not been built**. The Dockerfile and compose file are provided but unverified — and the README/TESTING docs say exactly that. If asked: "I wrote the packaging and documented that it's unverified; building it was impossible in my environment, and I don't claim otherwise."

Deployment options to discuss

- **Streamlit Community Cloud** — free, connects to the GitHub repo, but the 5.4 MB model must be in the repo or fetched at runtime (gitignored today — decision needed before deploying).

- **Any VM + Docker** — the compose file becomes the deployment unit; Supabase stays as the shared database.
- **HF Spaces** — Docker-space hosting that also fits this layout.

12. Roadmap — what comes next

Full version in `docs/roadmap.md`. The ordered short-list:

1. **Live Supabase round-trip** — create the project, run `supabase/schema.sql`, paste secrets, save and read back a real inspection (the one blocked manual step).
2. **Re-split the dataset by source image** — kills the near-duplicate leakage; retrain and report honest before/after metrics.
3. **GPU training run** — 640 px / 60 epochs via the provided Colab notebook; expect a real mAP50 jump.
4. **Docker build verification** — build and push the image once Docker is available.
5. **Class-balancing experiments** — oversample weak classes (SingleHotSpot/SingleDiode) or tune per-class confidences.
6. **Auth + multi-user RLS** — Supabase Auth, policies scoped to user ids, no more anon writes.
7. **Model versioning** — record every training run (git hash, data version, metrics) so `model_version` in the DB becomes a real lineage.
8. **Batch folder upload + PDF report** — inspector-day ergonomics.

13. Interview Q&A bank (45 questions)

Model answers, sized for spoken delivery. Numbers in brackets are the real measured values.

A. The project pitch

1. Explain this project in 60 seconds.

Solar panels develop faults that are invisible in daylight but glow in thermal images. Inspectors analyse those images by hand. SolarSight is a Streamlit web app: upload a thermal image or video, a fine-tuned YOLO11n model [5.4 MB] draws labelled boxes around 8 fault classes in [76 ms] on a CPU, and the inspection is saved to Supabase. A History screen shows past inspections with charts and CSV export. I built everything on a CPU-only laptop: dataset [7,339 images, Kaggle, CC BY-NC], training [10 epochs, ~94 min], evaluation [test mAP50 0.641], 33 passing tests, docs, and Docker packaging.

2. Why is this worth building — doesn't the camera software already do it?

Camera tools flag hot pixels but rarely classify the fault type. Classification by class (diode vs bypass vs string fault) drives different maintenance actions. Also, a database of classified inspections turns one-off images into a queryable maintenance history — that's the part manual review never gives you.

3. What was the hardest problem, and how did you solve it?

Two honest answers. Engineering: the ultralytics per-class metric arrays are ordered by position among classes *present* in the split, not by global class id — my evaluate script crashed with IndexError; fixed by indexing via `present.index(class_id)`. Methodology: the dataset's augmented near-duplicates leak across train/valid/test, inflating metrics — I can't fix the official split without breaking comparability, so I documented the caveat and put re-splitting on the roadmap.

B. Dataset questions

4. Where did the data come from and are you allowed to use it?

Kaggle `pkdarabi/solarpanel`, the ThermoSolar-PV anomaly dataset, version 49, license CC BY-NC 4.0 — non-commercial use with attribution. Fine for a portfolio project; not for a paid product.

5. Describe the dataset.

7,339 thermal JPGs at 640×640, YOLO-format labels, official splits 6,924/400/15 with 19,845/1,119/31 boxes, 8 fault classes. Three classes have no test ground truth.

6. What is YOLO label format?

One .txt per image; each line: `class_id x_center y_center width height`, all coordinates normalised 0–1 relative to image size.

7. Why is the test split a problem?

Two reasons: it's tiny (15 images, 31 boxes) so per-class numbers are fragile, and augmented near-duplicates of training images sit in it, so metrics are optimistic. Both are documented next to every reported number.

8. How would you fix the leakage?

Group-level split: hash source-image IDs, assign all augmented versions of a source to the same split, retrain, report before/after. I'd expect metrics to drop — and that lower number would be the honest one.

9. Is class imbalance an issue?

Yes — and it shows in results: well-represented, visually distinct classes like MultiDiode hit [0.995 mAP50] while SingleHotSpot [0.165] and SingleDiode [0.375] lag. Fixes: oversampling weak classes, per-class thresholds, more data.

C. Training questions

10. Walk me through your training run.

YOLO11n COCO-pretrained, fine-tuned 10 epochs at 320 px, batch 8, 2 workers, seed 42, on CPU [~94 min, torch 2.14.0+cpu]. Validation mAP50 climbed [0.127 → 0.554] every epoch with no plateau; best checkpoint exported to models/best.pt.

11. Why nano? Why 320 px? Why 10 epochs?

All the same constraint: CPU-only hardware. Nano is the smallest YOLO11; 320 px is ~4× cheaper per image than 640; 10 epochs fit a ~90-minute budget and the still-rising curve shows the pipeline works. The scale-up path (640 px, 60 epochs, GPU) is scripted in colab_train_gpu.ipynb — not yet run.

12. What is transfer learning and why use it?

Starting from weights pretrained on COCO and fine-tuning on thermal data. The early layers already encode edges/textures, so I need less data and less compute for good results.

13. What is a loss, and what does YOLO's loss contain?

The training objective to minimise. YOLO: box regression loss (box placement), classification loss (label correctness), DFL (box-edge distribution quality). All three fell monotonically across my 10 epochs.

14. How do you know it didn't overfit?

Validation mAP rose every epoch while training loss fell — no divergence. Also the test set was touched once, after training. True overfitting risk appears with longer training; that's when early stopping matters.

15. What does seed 42 do?

Fixes the random number generator (weight init, augmentation, shuffling) so runs are reproducible. Combined with pinned dependency versions and dataset version 49, someone can approximately reproduce my run.

16. How would you improve the model with one more week?

1) GPU run at 640 px / 60 epochs (scripted). 2) Leakage-free re-split, retrain, honest metrics. 3) Tinker with augmentation and per-class sampling for the weak classes. 4) Conf-matrix-driven error analysis.

D. Evaluation questions

17. Explain mAP50 vs mAP50-95.

A detection counts as correct if its IoU with truth ≥ threshold. mAP50 uses 0.50 — "found it, roughly right place." mAP50-95 averages over thresholds 0.50–0.95 in 0.05 steps — rewards precise boxes. Mine: [0.641] vs [0.541]; the gap is normal, the stricter metric is always lower.

18. Your precision is 0.692. What does that mean for a user?

Roughly 7 of 10 drawn boxes are real faults; 3 in 10 are false alarms the inspector dismisses. For an inspection-aid tool that's acceptable — humans verify; the model must not miss faults, so recall [0.762] matters more, and raising it is roadmap work.

19. Why are three classes shown as "—" instead of 0?

They have zero ground-truth instances in the test split, so precision/recall/AP are undefined — there is nothing to be right or wrong about. Printing 0.0 would falsely assert "tested and failed."

20. Why is SingleHotSpot bad, and what would you do?

[mAP50 0.165, precision 0.167, recall 0.500]. Few test boxes, small subtle hot regions easily confused with noise, and class imbalance. Actions: more data, oversampling, higher input resolution, error analysis on the confusion matrix.

21. What do the confusion matrix and PR curves tell you?

Confusion matrix: diagonal = correct; off-diagonal shows which classes get swapped (visually similar diode vs hotspot variants). PR curves: per-class precision-recall trade-off; area under them = AP; curves hugging the top-right are healthy. Both are saved in docs/assets.

22. Would you deploy this model?

As an assist tool, with disclosure: metadata-only metrics are optimistic due to dataset leakage; three classes were never tested; weak classes need a human check. As an autonomous decision system, no.

E. App & software questions

23. Why Streamlit?

Pure-Python web UI — no JS — ideal for internal tools and demos. Trade-offs I accept and can name: full-script reruns on every interaction, limited styling, one process per app. Caching neutralises the rerun cost for the heavy model.

24. Explain cache_resource vs cache_data.

cache_resource: one shared object across sessions — my model, loaded once per process. cache_data: memoises function outputs keyed by args — dataframes, downloads. Wrong choice = per-session reloads or shared mutable state bugs.

25. What happens on each Streamlit interaction?

The script re-runs top to bottom; widgets keep their values; cached resources are reused; session_state preserves per-user results between reruns.

26. Walk me through upload → saved inspection.

Sidebar sets site + thresholds → file uploaded → cached model predicts (or tracks video) → results become frozen Detection dataclasses → confidence filter [0.40 inclusive] → draw + table → Save: insert inspections row, bulk-insert detections rows → History can read it back.

27. Why is all Supabase code in one file?

src/database/db.py is the database layer. One seam to mock in tests, one place to swap providers, no SQL/network calls scattered in UI code. The class takes an injected client — dependency injection.

28. What is dependency injection and why here?

Instead of a function creating its own dependencies, they're passed in. InspectionsDB(client) gets a real Supabase client in production and a fake in tests — offline, instant, deterministic. 33 tests run in [~2.5 s] because of this.

29. How does the app behave without Supabase configured?

is_configured() checks secrets without raising; the save section shows a setup notice instead of crashing. Detection still works — storage is optional by design. Tested in CI by importing the app with no secrets.

30. How does video mode differ from image mode?

model.track(persist=True) adds ByteTrack tracking — stable track IDs across frames. Per-frame detections are deduplicated per track so one fault = one saved row (max confidence), not hundreds of frames.

31. Why conf 0.40 and IoU 0.45?

0.40 balances missed faults vs false-alarm flood (chosen during evaluation; visible in the PR trade-off). NMS IoU 0.45 (vs ultralytics default 0.7) keeps close-together faults from being merged into one box. Both live in config.py, tunable live from the sidebar.

32. What would break at 1,000 concurrent users?

Streamlit's model: one process, script reruns per interaction — it's an internal-tool framework. Fix path: real frontend + FastAPI inference service (batched), managed Postgres stays; model serving via TorchScript/ONNX or Triton. The layered code makes that extraction cheap — UI, pipelines, DB are already separate.

F. Database questions

33. Why a database at all?

Analyses must outlive the session and be queryable: filter by site, chart fault types over time, export CSV for reports. That's relational data — inspections 1—N detections.

34. Why Supabase over SQLite/MySQL/Firebase?

SQLite: local file, no multi-user API. MySQL: fine DB, but Supabase adds hosted Postgres + instant REST API + RLS. Firebase: NoSQL, weak fit for relational filters/aggregations. For a portfolio app, zero server maintenance matters too.

35. Explain the schema.

inspections: uuid PK, site_name, source_filename, media_type (CHECK image/video), model_version, created_at (indexed DESC), total_faults. detections: uuid PK, FK inspection_id ON DELETE CASCADE, class_name (indexed), confidence (CHECK 0–1), x1..y2, nullable track_id. Three indexes, each mapped to one real query.

36. What is RLS and why do you need it with an anon key?

Row Level Security: rules evaluated per query inside Postgres defining which rows a role may touch. The anon key ships inside the app, so assume it leaks; without RLS, anyone could read/modify everything — and Supabase auto-exposes a REST API. My policies allow anon insert/select/delete (demo trade-off); production would scope rows to authenticated users.

37. Why indexes on those three columns?

created_at DESC → History's newest-first list; inspection_id → drill-down lookups per inspection; class_name → fault-type chart aggregation. An index on an unused column is pure write cost — every index here maps to a query the app actually runs.

38. Why store metadata and not the images?

DB rows stay tiny; no blob storage costs; no privacy burden of retaining site imagery; the upload is analysed in memory and discarded. If images were needed later: Supabase Storage + a URL column, not the DB.

39. Your save isn't transactional — is that a problem?

Yes, a known one: PostgREST has no client-side transactions, so a failure between the parent insert and detections bulk-insert could leave a childless parent. Documented in TESTING.md. Fixes: a Postgres RPC function wrapping both inserts, or a cleanup job for orphan parents.

G. Testing & process questions

40. What do your 33 tests cover and what not?

Covered: pipeline filtering/conversion, track dedupe/summaries, all db.py methods against a fake Supabase client, detection→row mapping, two AppTest smoke tests booting the real app. Not covered: the real network round-trip (no live Supabase on this machine) and real model accuracy inside the app (the model is evaluated separately offline). Saying what tests don't cover is part of testing honestly.

41. Fake vs mock?

Both are test doubles. A mock asserts interactions (was this called?); a fake is a working lightweight implementation. My FakeSupabaseClient reproduces the fluent chain semantics, so tests assert behaviour, not call order.

42. Tell me about a test that caught a real bug.

The AppTest smoke test failed when located by radio index — the Upload screen adds its own "Media type" radio when a model exists, shifting indices. Fix: locate by label "Screen". Lesson: UI locators should anchor on stable labels, not layout positions. Seven more documented in TESTING.md.

43. What does CI do here?

GitHub Actions on every push: pip check (dependency sanity), import-the-app-without-secrets (proves graceful degradation), pytest. Same steps locally via scripts/ci_local.sh — all green on this machine.

H. Honest-limitations questions (they WILL ask)

44. What are you NOT claiming?

Precisely: (1) metrics are optimistic — augmented near-duplicates leak across splits; (2) three of eight classes have no test ground truth; (3) the Supabase round-trip is tested offline only — no live project yet; (4) the Docker image is written but never built (no Docker on this machine); (5) 10 epochs / 320 px is a proof-of-capability run — the 640 px GPU notebook is provided, not run; (6) parts of the codebase were AI-assisted; every claim in the docs traces to a run I executed.

45. What would you do differently?

1) Re-split by source image from day one — it changes every reported number. 2) Start the Supabase project earlier so the full loop is exercised live. 3) Verify Docker in a VM. 4) Log per-class validation metrics during training to catch weak classes sooner.

14. Known limitations — the honest list

Limitation	Detail	Status / fix
Optimistic metrics	Augmented near-duplicates across splits (3 versions per source image)	Documented; re-split by source image on roadmap
Untested classes	MultiByPassed, StringOpenCircuit, StringReversedPolarity absent from test GT	Shown as "—", never 0
Weak classes	SingleHotSpot [0.165], SingleDiode [0.375] mAP50	Error analysis + rebalancing on roadmap
Small model / short run	10 epochs @ 320 px on CPU	GPU notebook (640 px / 60 ep) provided, not run
Supabase not exercised live	No project/keys on this machine; schema + graceful degradation + 33 offline tests exist	User manual step; blocked, documented
Docker unverified	Docker not installed; Dockerfile + compose written, healthcheck on /_stcore/health	Documented as unverified in TESTING.md
Non-transactional save	PostgREST client-side limit; orphan-parent risk	Documented; RPC-based fix identified
Dataset license	CC BY-NC 4.0 — non-commercial only	Attribution in DATA.md/README
AI-assisted build	Portions of the codebase were built with AI assistance	Disclosed in README; all claims trace to executed runs

15. Cheat sheet — every number you must remember

Fact	Value
Dataset images / size	7,339 · 640×640 thermal JPGs
Splits (images / boxes)	6,924 / 400 / 15 — 19,845 / 1,119 / 31
Classes	8 (3 absent from test GT)
Test overall	P 0.692 · R 0.762 · mAP50 0.641 · mAP50-95 0.541
Best / worst class (mAP50)	MultiDiode 0.995 · SingleHotSpot 0.165
Valid mAP50 curve	epoch 1: 0.127 → epoch 10: 0.554 (mAP50-95 0.368)
Training run	yolo11n · 10 ep · 320 px · batch 8 · seed 42 · CPU · ~94 min
Model file	models/best.pt · 5.4 MB · conf 0.40 · IoU 0.45
CPU speed	76 ms/image @640px (nano, measured)
Tests	33 passing · ~2.5 s
Stack versions	Python 3.11.16 · torch 2.14.0+cpu · ultralytics 8.4.165 · streamlit 1.64.0 · supabase 2.31.0 · opencv 5.0.0
App check	GET /_stcore/health → ok (verified on :8601)
Dataset license	CC BY-NC 4.0 · Kaggle pkdarabi/solarpanel v49

16. Glossary quick reference

Term	One-line definition	Where in this project
YOLO11n	Smallest YOLO11 one-pass object detector	The model; trained to models/best.pt
mAP50	Mean AP at IoU 0.50 across classes	Headline metric: 0.641 on test
IoU	Box overlap = intersection ÷ union	Detection correctness + NMS threshold 0.45
Precision / Recall	Boxes that were real / faults that were found	0.692 / 0.762 on test
NMS	Drops duplicate overlapping boxes	Inside ultralytics inference
Transfer learning	Fine-tune a pretrained model	COCO-pretrained yolo11n → thermal
Epoch	One pass over training data	10 epochs, ~9 min each on CPU
Overfitting	Memorising train, failing on new data	Not observed: valid mAP rose every epoch
ByteTrack	Assigns stable IDs to objects across frames	Video pipeline, model.track(persist=True)
Streamlit	Python→web-app framework, reruns script per interaction	The whole UI, app.py
st.cache_resource	Load once per process, share across sessions	Model loading
Supabase	Hosted Postgres + auto REST API + RLS	Inspections storage
RLS	Per-row access rules enforced by the DB	Anon policies in schema.sql
Foreign key / cascade	Child rows tied to parent; auto-deleted	detections → inspections
Index	Structure that speeds specific queries	created_at, inspection_id, class_name
pytest	Python test framework	33 tests in tests/
Test fake	Working stand-in for a dependency	FakeSupabaseClient in conftest
Dependency injection	Pass dependencies in instead of creating them	InspectionsDB(client)
CI	Automated checks on every push	.github/workflows/ci.yml
Docker image / container	Frozen app snapshot / running instance	Dockerfile, docker-compose.yml (unverified)

HEALTHCHECK	Probe that the service is alive	GET /_stcore/health → ok
CSV	Comma-separated table export	History screen export
conda env	Isolated Python installation	solarsight (Python 3.11.16)
seed	Fixed RNG start → reproducibility	42 in training

SolarSight — The Complete Beginner's Guide. Generated from the project's real artifacts (docs/, training outputs, test suite). Every metric was measured on the author's CPU-only machine; items not executed are labelled as such. Companion documents: README.md, docs/DATA.md, TRAINING.md, TESTING.md, WALKTHROUGH.md, INTERVIEW_PREP.md, architecture.md, roadmap.md.